

Cours 7

Programmation orientée objet

Jérémy Kairis

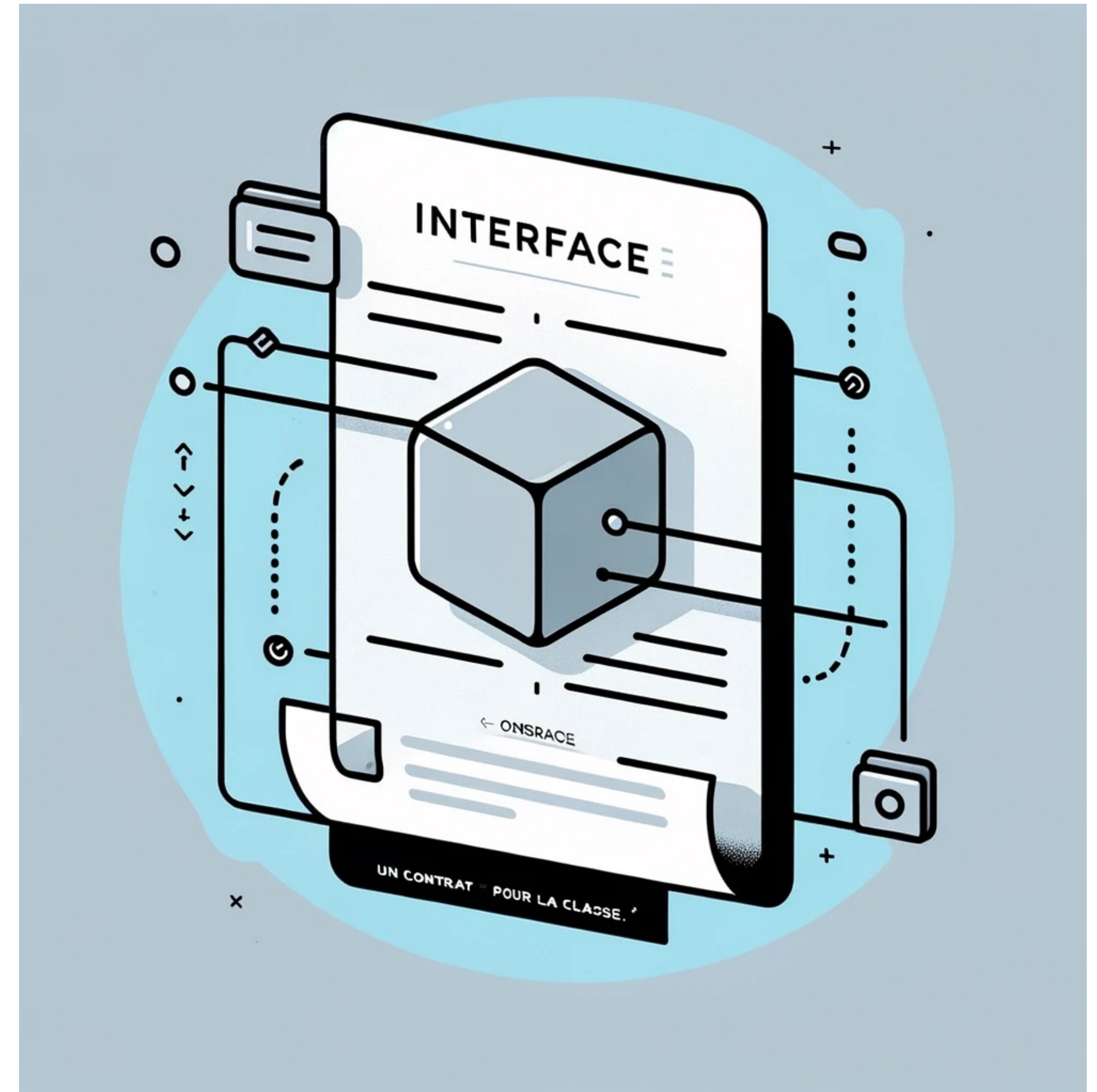
Les interfaces

Les interfaces

Définition

Une interface peut être comparée à un contrat.

Elle définit un ensemble de méthodes et propriétés que toute classe qui l'implémente doit respecter, agissant comme un contrat qui assure une structure commune sans imposer d'implémentation spécifique.



Les interfaces

Déclaration

Les interfaces sont définies à l'aide du mot clé « ***interface*** » et, en général, leur nom commenceront par un « **I** ».

Il y a deux règles à respecter lors de la déclaration des interfaces :

- Une interface ne peut pas contenir de champs (variables membres).
- Les membres (propriétés, méthodes...) d'interface sont automatiquement « ***public*** ».

De plus, aucun bloc d'instruction n'est généralement spécifié.

```
public interface IDessinable
{
    void Dessiner();
}
```

Les interfaces

Implémentation

Lorsque nous disons qu'une classe ou une structure implémente une interface, cela signifie que la classe ou la structure s'engage à implémenter tous les membres définis par l'interface.

Les interfaces

:

Pour signifier à une classe qu'elle doit implémenter l'interface, nous allons le faire comme pour l'héritage avec “:”.

De plus, C# nous autorise d'implémenter plusieurs interfaces à une même classe en plus de l'héritage de classe en lui-même séparé par des “, ”.

```
public interface IDessinable
{
    void Dessiner();
}
```

```
public class Cercle : IDessinable
{
    public void Dessiner()
    {
        Console.WriteLine("Dessin d'un cercle");
    }
}
```


Les interfaces

Utilisation

Dans le cas de l'abstraction de code, nous allons nous servir des interfaces afin de déclarer les fonctionnalités que devront déclarer nos classes.

Enfin, nous utiliserons le polymorphisme afin de pouvoir déclarer nos variables en utilisant le type de l'interface afin d'être totalement indépendant du type de la variable réelle et du code réellement exécuté en arrière plan.

Cela sera utile pour abstraire le code lors des tests unitaires.

```
DessinerEnConsole(new Carre());  
DessinerEnConsole(new Cercle());
```

```
void DessinerEnConsole(IDessinable dessinable)  
{  
    Console.WriteLine(dessinable.Dessiner());  
}
```

```
public interface IDessinable  
{  
    string Dessiner();  
}  
public class Cercle : IDessinable  
{  
    public string Dessiner() => "Dessin d'un cercle";  
}  
public class Carre : IDessinable  
{  
    public string Dessiner() => "Dessin d'un carré";  
}
```

Les interfaces

Garbage collector

Le « garbage collector » recherche les objets qui ne sont plus utilisés par l'application.

S'il considère qu'un objet est candidat à la destruction, il appelle le destructeur (s'il y a lieu) et libère la mémoire utilisée pour stocker l'objet.

Cependant, il est impossible de prévoir lorsqu'un ramassage se produira.



Les interfaces

IDisposable

Cependant, le « garbage collector » n'a pas connaissance des ressources non managées, telles que les fichiers et flux ouverts.

Nous devons donc gérer nous même la libération des ressources prises par ces objets.

A cette fin, nous utiliserons l'interface « ***IDisposable*** », qui nous imposera de déclarer une méthode « ***Dispose()*** », afin d'y implémenter le code de la libération des ressources.

Lorsque nous n'utilisons plus l'objet, il nous suffit d'appeler cette méthode afin de rendre cet objet 'collectable' par le « garbage collector ».

Cette interface est déjà implémentée par les classes travaillant sur les flux en général. (tel que les fichiers, les connexions aux bases de données etc.)

```
class ExempleDisposable : IDisposable
{
    public void Dispose()
    {
        // Libérer les ressources non managées ici.
        // Par exemple, se désabonner à un événement.
    }
}
```

Les interfaces

using

Afin de simplifier notre code et d'appeler la méthode « ***Dispose*** » de manière automatique, nous pouvons déclarer un bloc d'instruction

« ***using*** ».

Ce bloc nous permet de déclarer une variable de type « ***IDisposable*** », qui ne sera accessible que dans ce bloc d'instruction.

Une fois que le bloc d'instruction se termine, la méthode « ***Dispose*** » est appelée automatiquement.


```
using (var lecteur = new StreamReader("exemple.txt"))
{
    // Lecture du contenu du fichier ligne par ligne
    string ligne;
    while ((ligne = lecteur.ReadLine()) != null)
    {
        Console.WriteLine(ligne);
    }
} // Une fois que la lecture est terminée, le fichier est automatiquement fermé.
```

Exercice

Banque

12. Définir l'interface « IAccount », afin de limiter l'accès à consulter la propriété « Balance » et d'utiliser les méthodes « Depotsit » et « Withdraw ».

13. Définir l'interface « IBankAccount » ayant les mêmes fonctionnalités que « IAccount ».

Elle lui permettra, en plus, d'invoquer la méthode du « ApplyInteret » et offrira un accès en lecture au « Owner » et au « Number ».

Les constructeurs

Les constructeurs

new

Les constructeurs sont des méthodes particulières de la classe qui sont exécutées à l'appel de l'opérateur « ***new*** ».

Les constructeurs portent le même nom que la classe, mais n'ont pas de type de retour, et servent habituellement à initialiser les données membres du nouvel objet.

De plus, les constructeurs n'appartiennent qu'à la classe qui les déclare et ne sont pas, par conséquent, hérités par les classes dérivées.

Enfin, ceux-ci ont les mêmes niveaux d'accessibilité que n'importe quel membre d'une classe.

```
var p = new Personne();
```

```
class Personne
```

```
{
```

```
    public Personne() { }
```

```
}
```


Les constructeurs

Par défaut

Si aucun constructeur n'est défini, C# en créera un par défaut quiinstanciera l'objet.

Ce constructeur par défaut est un constructeur sans paramètre.

A partir du moment où nous fournissons un constructeur, quel qu'il soit, ce constructeur par défaut disparaît.

```
var p = new Personne("Kairis", "Jérémy");
```

```
class Personne
```

```
{
```

```
    public Personne(string nom, string prenom) { }
```

```
}
```

Les constructeurs

Surcharge

Bien que spéciaux, les constructeurs restent des méthodes.

Par conséquent ceux-ci peuvent être surchargés en respectant le concept de surcharge de méthodes.

Le but étant d'offrir plusieurs possibilités de construire les objets.


```
var p = new Personne("Kairis", "Jérémy");  
var p1 = new Personne(33);
```

```
class Personne  
{  
    public Personne(string nom, string prenom) { }  
  
    public Personne(int age) { }  
}
```

Les constructeurs

Héritage

Dans le cadre de l'héritage, afin de créer un objet en mémoire, le « runtime » va créer une suite d'objet en respectant l'ordre défini par l'héritage. Pour parvenir à ce résultat, le « runtime » va partir du constructeur de la classe instanciée et appeler chaque constructeur des classes parentes jusqu'à la classe “ ***Object*** ”.

Une fois arrivé à ce dernier, il va exécuter le corps des constructeurs rencontrés en redescendant vers le constructeur initialement appelé.


```
var p = new Personne("Kairis");  
var p1 = new Employe("Doe", "Prof");
```

```
public class Personne  
{  
    public string Nom { get; set; }  
  
    public Personne(string nom)  
    {  
        Nom = nom;  
    }  
}  
public class Employe : Personne  
{  
    public string Poste { get; set; }  
  
    public Employe(string nom, string poste) : base(nom)  
    {  
        Poste = poste;  
    }  
}
```

Les constructeurs

base

Par défaut, lorsque rien n'est spécifié, le « runtime » appelle implicitement le constructeur sans paramètre de la classe parente.

Ce qui veut dire que s'il n'existe pas de constructeur sans paramètre, nous devons utiliser le mot-clé « **base** » suivi de parenthèses et des paramètres nécessaires après la déclaration du constructeur de la classe enfant suivi de “ : ”.

```
var p = new Personne("Kairis");  
var p1 = new Employe("Doe", "Prof");
```

```
public class Personne(string nom)  
{  
    public string Nom { get; set; } = nom;  
}
```

```
public class Employe(string nom, string poste) : Personne(nom)  
{  
    public string Poste { get; set; } = poste;  
}
```


Les constructeurs

this

Puisque nous pouvons appeler un constructeur de la classe parent, nous pouvons également appeler un autre constructeur de la classe courante.

Pour ce faire, le mot-clé « **base** » est remplacé le par « **this** ».

```
var e1 = new Example();  
var e10 = new Example(10);
```

```
class Example  
{  
    public int valeur;  
  
    public Example()  
    {  
        valeur = 1;  
    }  
  
    public Example(int valeur) : this()  
    {  
        this.valeur += valeur;  
    }  
}
```

Les constructeurs

this

Il arrive qu'une variable membre et une variable locale portent le même nom.

Le compilateur est tout à fait capable de faire la distinction entre les deux.

Dans ce cas, la variable locale “dissimule” la variable membre.

Pour l'atteindre, on utilise le mot-clé “***this***”.

Exercice

Banque

14. Ajoutez, dans la classe « Account », deux constructeurs prenant en paramètre :
 - Le numéro et le titulaire
 - Le numéro, le titulaire et le solde (pour le cas d'une base de données)
15. Le cas échéant, ajoutez le ou les constructeurs aux classes « CurrentAccount » et « SavingsAccount ».
16. Changer l'encapsulation des propriétés des classes « Person », « Account » et « SavingsAccount » afin de spécifier leur mutateur en « private ».
17. Définir ce qu'il manque pour que le programme continue à tourner.

Les exceptions

Les exceptions

Introduction

La gestion des erreurs est une partie importante du développement, que ce soit en importance ou en temps.

Par défaut, lorsqu'une erreur survient, celle-ci va remonter niveau par niveau jusqu'à atteindre le point d'entrée de l'application.

Si entre temps, nous ne la capturons pas pour la traiter, le système d'exploitation mettra fin à votre application en vous signalant qu'une erreur n'a pas été gérée.

En C#, une telle erreur est appelée « ***Exception*** ».

Il existe une multitude de types d'erreur mais tous, au final, héritent de la classe « ***Exception*** ».

Unhandled exception. System.NullReferenceException:

Object reference not set to an instance of an object.

at Program.<Main>\$(String[] args) in /Users/Jeremy/IFOSUP/2024 - 2025/

Bank/Program.cs:line 62

Les exceptions

try ... catch ... finally

Lorsqu'une exception est levée, celle-ci met fin au bloc d'instruction courant (méthode, if, switch, try, etc.) et commence à remonter jusqu'à la racine de l'application.

Pour intercepter une exception nous devons utiliser le bloc « ***try*** ». Celui-ci doit être suivi d'au moins un bloc « ***catch*** » ou d'un bloc « ***finally*** ».

Si notre bloc « ***try*** » détecte une exception, celui-ci s'interrompt et envoie l'erreur en paramètre du bloc « ***catch*** » (si existant).

Le bloc « ***finally*** » (si existant) s'exécute quoi qu'il arrive et est le plus souvent utilisé pour nettoyer les ressources utilisées.


```
try
{
    // Code pouvant générer une exception
    int a = int.Parse(Console.ReadLine());
    int b = int.Parse(Console.ReadLine());
    int résultat = a / b;
}
catch (DivideByZeroException ex)
{
    // Gestion de l'exception DivideByZeroException
    Console.WriteLine("Détails de l'exception division par zéro: " + ex.Message);
}
catch (Exception ex)
{
    // Gestion des autres exceptions
    Console.WriteLine("Une erreur s'est produite : " + ex.Message);
}
finally
{
    // Code exécuté qu'une exception se produise ou non
    Console.WriteLine("Fin du programme.");
}
```

Les exceptions

throw

L'instruction « ***throw*** » sert à signaler la présence d'une situation anormale (exception) pendant l'exécution du programme.

C'est grâce à lui que nous allons déclencher des exceptions.

Le mot-clé « ***throw*** » peut aussi servir à renvoyer une exception déjà existante afin, par exemple, de réaliser des actions localement et de renvoyer à l'appelant l'exception par la suite.

```
try
{
    throw new NotImplementedException("En cours de développement.");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    throw;
}
```


Les exceptions

Exception

Pour créer ses propres exceptions, il n'y a qu'une règle à respecter.

La classe que nous allons définir doit être une classe dérivée de la classe
« *Exception* ».

```
throw new MonException("En cours de développement.");
```

```
public class MonException : Exception  
{  
    public MonException(string message) : base(message)  
    {  
        Console.WriteLine(message);  
    }  
}
```


Exercice

Banque

18. Dans la classe « Account » :

- Au niveau de la méthode « Deposit », déclenchez une exception de type « `ArgumentOutOfRangeException` » si le montant n'est pas supérieur à 0 (zéro).
- Faites de même au niveau de la méthode « Withdraw » et y ajouter le déclenchement d'une exception de type « `InsufficientBalanceException` » si le montant ne peut être retiré.

19. Au niveau de la classe « CurrentAccount » :

- Au niveau de la propriété « CreditLine », déclenchez une exception de type « `ArgumentOutOfRangeException` » si la valeur n'est pas supérieur ou égale à 0 (zéro).